

Warm up

Last minute questions on Assignment 5? How is Assignment 6 going? Any other questions or concerns about the logistics and final exam?

Precept 22: Writing instructions and the Assignment 6 “A” attack

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

April 21, 2026

Overview

- 1 The “A” attack
- 2 Composing assembly instructions
- 3 Summary

Recall: grader.c

- Compiled with:
`gcc217 -O -fomit-frame-pointer grader.c -o grader`
 - `-O` (capital “oh”): the compiler may make optimisations that differ from what you would expect
 - ◇ Make sure to read the assembly code; do not assume the contents of memory or the stack frame
 - `-fomit-frame-pointer`: the compiler does not use a frame pointer register (e.g., `x29`)
- The program reads a string from stdin into a global name buffer in the bss section
- `mprotect((void*)((unsigned long)name & 0xffffffffffff000), 1, PROT_READ | PROT_WRITE | PROT_EXEC);`
 - Change the permissions of the memory page containing `name` (in bss) to allow execution
 - This allows us to execute code that we write into `name` (e.g., shellcode)
 - Program is intentionally and artificially susceptible to an attack
 - “B” attack would work without this, but “A” attack would not

Recall: the “B” attack

- Goal: provide a string input that causes the program to give you a “B” grade (as opposed to the default “D”)
- Observe: `readString` keeps reading from `stdin` until it reads EOF or newline
 - It does not check if the input exceeds the size of `buf`
- Insight: write beyond the end of the `buf` array to overwrite the saved value of `x30` in `getName`’s stack frame
- Where should we overwrite `x30` to point to?
 - Can we jump to the assignment of `grade = 'B'`?
- Approach: use the memory map to determine:
 - the address of the instruction that assigns `grade = 'B'`
 - how many bytes to write in order to overwrite `x30`

The “A” attack

- Goal: provide a string input that causes the program to give you an “A” grade
- Observe: can’t jump to the code that assigns `grade = 'A'` because it is not in the program
- Insight: write our own code that assigns `grade = 'A'`
- Where can we write this code?
 - We can write it into the `name` buffer in `bss` (executable due to the `mprotect` call)
- What instructions should we write?
 - The instructions should assign `'A'` to the memory address of `grade`

```
mov w0, 'A'
adr x1, grade
strb w0, [x1]
b (call of printf in main())
```
- How do we tell the program to execute these instructions?
 - Similar to the “B” attack, we can overwrite `getName`’s `x30` to point to the address of the first instruction in `name`

Overview

1 The “A” attack

2 Composing assembly instructions

3 Summary

Recall: writing memory addresses using fwrite

To write the 8-byte address 0x0123456789abcdef:

```
FILE *psFile;  
psFile = fopen("data", "w");  
  
unsigned long ulData;  
ulData = 0x0123456789abcdef;  
fwrite(&ulData, sizeof(unsigned long), 1, psFile);  
  
fclose(psFile);
```

- fwrite dumps the exact byte representation of ulData from memory into the file, with no conversion or formatting
- This write is endianness-dependent
 - On a little-endian machine, the bytes of ulData will be written in reverse order (i.e., 0xef first, then 0xcd, etc.)
 - On a big-endian machine, the bytes of ulData will be written in the same order as they appear in the code (i.e., 0x01 first, then 0x23, etc.)
- Can use a similar approach to write instructions

Composing assembly instructions: overview

- Goal: write assembly instructions that assign `grade = 'A'`

```
gcc217 createdataA.c miniassembler.c -o createdataA
./createdataA
./grader.sh dataA
```

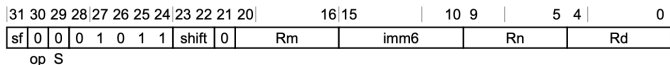
- The `createdataA` program must write 4 machine language instructions to `dataA`
- Rules
 - Each instruction is exactly 4 bytes long
 - Each instruction must begin at an address that is evenly divisible by 4
 - Each instruction is stored in little-endian byte order
- How does the processor interpret four bytes as instructions?
 - ARMv8 instruction set architecture (ISA) defines the encoding of instructions

Parts of an instruction

- Parts of an instruction: opcode, source registers, destination register, instruction-specific fields (e.g., immediate values, shift amounts, etc.)
 - Opcode: specifies the operation to be performed (e.g., add, subtract, load, store, etc.)
 - Source register(s): specifies which register(s) to read from
 - Destination register: specifies which register to write the result to
 - Why are the register portions of the instruction five bits long?
 - ◇ 32 registers in ARMv8 (including zero register) → 5 bits needed to uniquely identify each register (since $2^5 = 32$)
 - Instruction-specific fields: vary based on the type of instruction (e.g., immediate values for arithmetic instructions, offset values for load/store instructions, etc.)

Example: add x2, x0, x1

- Computes $\text{reg}[x2] = \text{reg}[x0] + \text{reg}[x1]$



32-bit variant

Applies when $\text{sf} == 0$.

ADD <Wd>, <Wn>, <Wm>{, <shift> #<amount>}

64-bit variant

Applies when $\text{sf} == 1$.

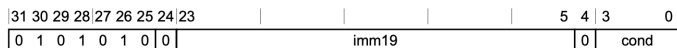
ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}

- $\text{sf} = 1$: 64-bit operation
- $\text{shift} = 00$: no shift applied
- $\text{Rm} = 00001$: second source register is x1
- $\text{imm6} = 000000$: no immediate value
- $\text{Rn} = 00000$: first source register is x0
- $\text{Rd} = 00010$: destination register is x2

- Put together: 1 0001011 00 0 00001 000000 00000 00010 $\rightarrow$ 0x8b010002

Another example: `blt label1`

- Conditionally branches to the address denoted by `label1`



19-bit signed PC-relative branch offset variant

`B.<cond> <label>`

- But, each address is 8 bytes long → how do we encode the target address in the instruction?
 - Express the target address as an offset from the current instruction's address (i.e., PC-relative addressing)
 - Compute the offset
$$[\text{address of label1}] - [\text{address of blt instruction}]$$
 - Address of each instruction is a multiple of 4 → offset is a multiple of 4 → no need to encode the last two bits of the offset (since they will always be 00)
 - Place the offset (divided by 4) in the instruction's `imm19` field

blt label1

- Suppose label1 is at address 0x00001230
- Suppose further that the blt instruction is at address 0x00004320
- Then, the offset is $0x00001230 - 0x00004320 = -0x\dots\text{ffffcf}10$
 - In binary: ... 1111 1111 1111 1111 1100 1111 0001 0000
 - imm19 = 1 1111 1100 1111 0001 00
- cond = 1011 for blt
- Put together: 01010100 1111111001111000100 0 1011 $\rightarrow$ 0x54fe788b

A MiniAssembler Module

- Manually encoding instructions is error-prone and tedious → we can write a mini-assembler module to automate the encoding process
- `setField` static helper for setting the bits of an instruction

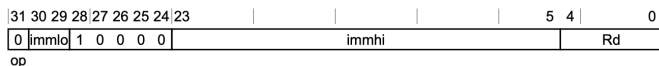
```
static void setField(unsigned int uiSrc,
                    unsigned int uiSrcStartBit,
                    unsigned int *puiDest,
                    unsigned int uiDestStartBit,
                    unsigned int uiNumBits) { ... }
```

- You need to implement this helper

- `MiniAssembler_adr` is implemented for you
- Your job: implement `MiniAssembler_mov`, `MiniAssembler_strb`, and `MiniAssembler_b` in `miniassembler.c`
- `createdataA.c` will then use these functions to write the instructions for the “A” attack
 - Can add additional functions for the “A+” attack
- The `testminiassembler.c` client code can be used to test your implementation against `testminiassembler.ref` reference

Your turn: `adr x1, label2`

- `adr` also uses PC-relative addressing
 - But does not divide by four as in `blt`, why? Because `label2` presumably holds data, which may not be 4-byte aligned
- Assume the offset between the current instruction and `label2` is
00000000 00000000 00110000 11110000
- What is the instruction encoding for `adr x1, label2`?



Literal variant

ADR <Xd>. <label>

- Rd = 00001: destination is x1
 - immlo = 00: last two bits of the offset are 00
 - immhi = 00000 00110000 111100: first 19 bits of the offset
- Put together: 0 00 10000 0000000110000111100 00001 → 0x10018781

Overview

- 1 The “A” attack
- 2 Composing assembly instructions
- 3 Summary

Summary

- Assignment 6: the “A” attack
- Composing assembly instructions and the mini-assembler
- Dates to remember:
 - Assignment 5: due Tue, 4/21 at 9pm (tonight!)
 - Assignment 5 Quiz: Wed, 4/22 in lecture (tomorrow!)
 - Assignment 6: due Wed, 4/29 at 9pm
 - Final exam: Sun, 5/10, 4-7pm in Friend 101
- Thank you for a great semester!!